

Conversion d'un document manuscrit en une version texte

CASANOVA KINKELA Lise
N°SCEI 22685

Positionnement thématique:

- Sciences industrielles (Traitement du signal)
- Informatique (Informatique pratique)

Problématique retenue:
Convertir un texte manuscrit en une version
numérique

Plan

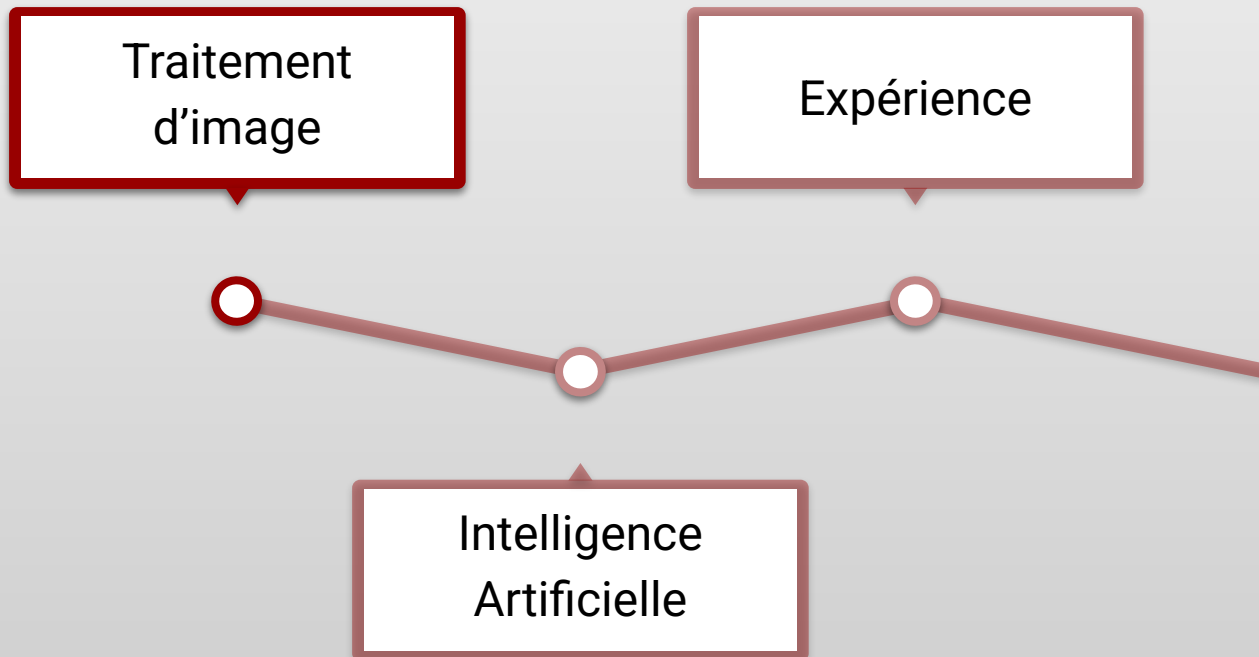
Traitement d'image (TI)

Intelligence artificielle (IA)

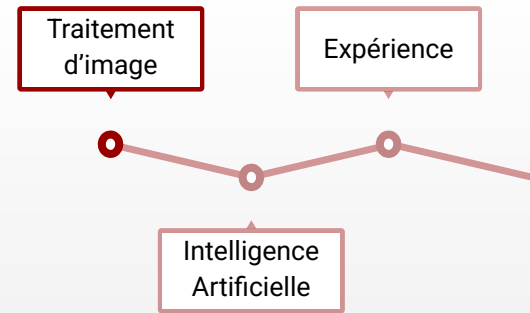
Expérience

Conclusion et discussion

Traitement d'image (TI)



Traitement d'image (TI)



Format adapté



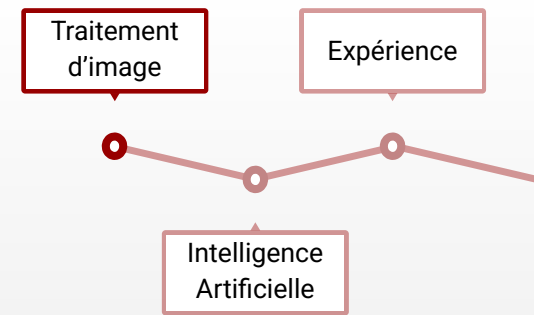
Image en
nuance de gris



Lisibilité du
texte

Validation des hypothèses

Traitement d'image (TI)



	Situation	Compression	Bibliothèque de traitement d'image
JPG	En sortie du téléphone	✓	✗
BMP	Entrée du traitement de texte	✗	✓

Traitement d'image (TI)

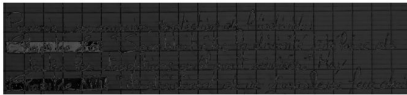
Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait revêues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit

Pouvoir

Nuance de gris

Moyenne (au programme)

Image display 'Teste2.bmp' in shades of grey (Simple average method)



Traitement
d'image

Expérience

Intelligence
Artificielle

R	G	B
---	---	---

$$Intensite = \frac{R+G+B}{3}$$

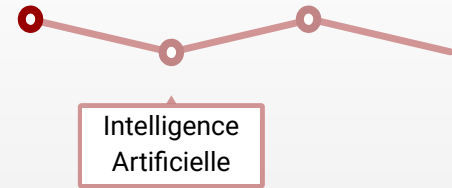
R	G	B
---	---	---

Intensite	Intensite	Intensite
-----------	-----------	-----------

Traitement d'image (TI)

Traitement
d'image

Expérience



Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait recueillies." May
Chapitre XVII: "ils décidaient de ne transférer leur droit

Pouvoir

Nuance de gris

Moyenne (au programme)

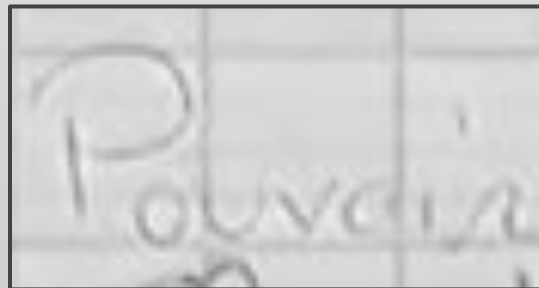
Image display 'Teste2.bmp' in shades of grey (Simple average method)



Maximum

Image display 'Teste2.bmp' in shades of grey (Maximum brightness method)

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait recueillies." May
Chapitre XVII: "ils décidaient de ne transférer leur droit



R

G

B

$$Intensite = \max(R, G, B)$$

R

G

B

Intensite

Intensite

Intensite

Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit

Pouvoir

Nuance de gris

Moyenne (au programme)

Maximum

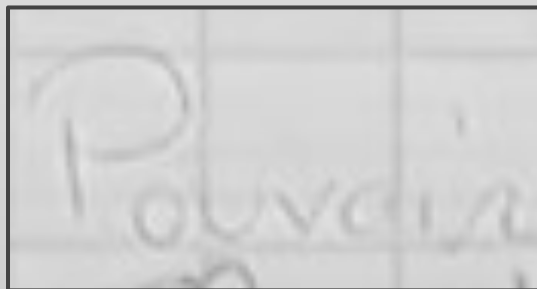
Luma

Image display 'Teste2.bmp' in shades of grey (Simple average method)



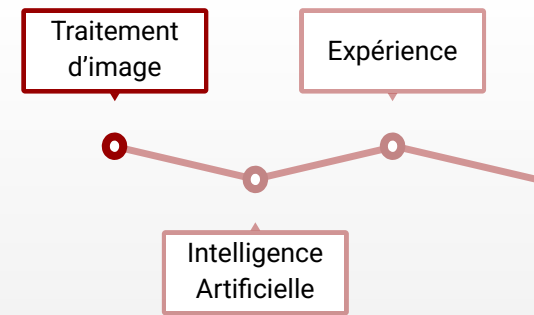
Image display 'Teste2.bmp' in shades of grey (Maximum brightness method)

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit



Traitement d'image (TI)

- Sensibilité de l'oeil humain
- Méthode des télévision



X : Composante représentant un mélange des sensibilités des cônes

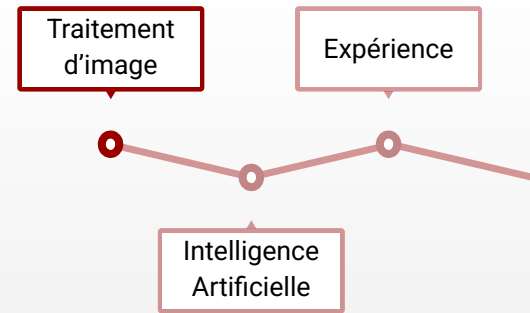
Y : Correspond exactement à la luminance perçue

Z : Représente principalement la sensibilité au bleu



Traitement d'image (TI)

- Sensibilité de l'oeil humain
- Méthode des télévision

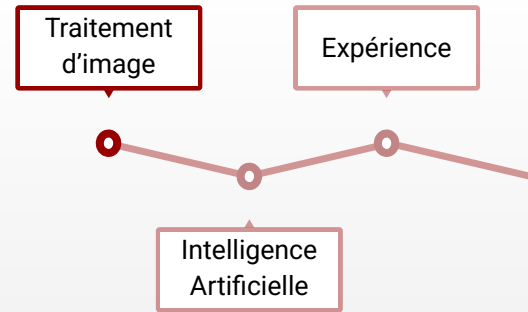


Référence du blanc : E (1, 1, 1)

	R	G	B
X	0.488	0.311	0.204
Y	0.176	0.813	0.011
Z	0	0.010	1.007

Traitement d'image (TI)

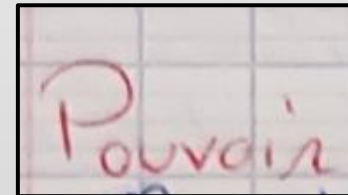
- Sensibilité de l'oeil humain
- Méthode des télévision



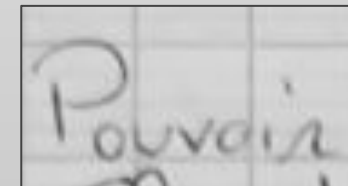
Référence du blanc : E (1, 1, 1)

	R	G	B
Y	0.176	0.813	0.011

Pouvoir : soumission, protection de l'individu :
Chapitre 10 : "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait révéleres." May
Chapitre XVII : "ils décidèrent de ne transférer leur droit



Pouvoir : soumission, protection de l'individu :
Chapitre 10 : "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait révéleres." May
Chapitre XVII : "ils décidèrent de ne transférer leur droit



Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

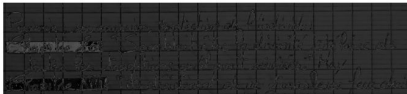
Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité faite à l'image de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "Ils décidèrent de ne transférer leur droit

Pouvoir

Nuance de gris

Moyenne (au programme)

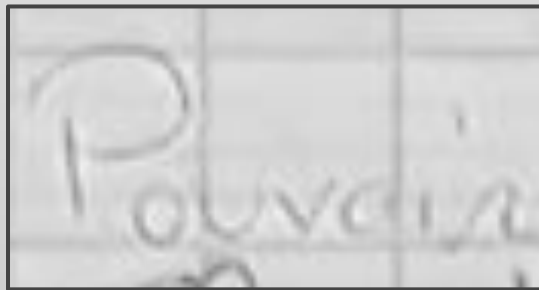
Image display 'Teste2.bmp' in shades of grey (Simple average method)



Maximum

Image display 'Teste2.bmp' in shades of grey (Maximum brightness method)

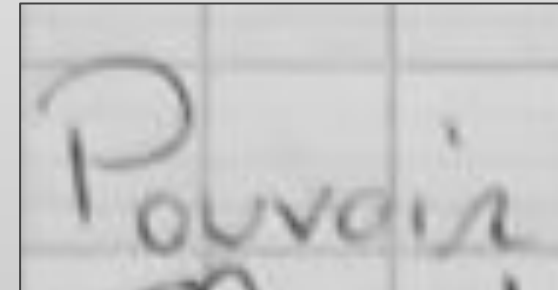
Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité faite à l'image de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "Ils décidèrent de ne transférer leur droit



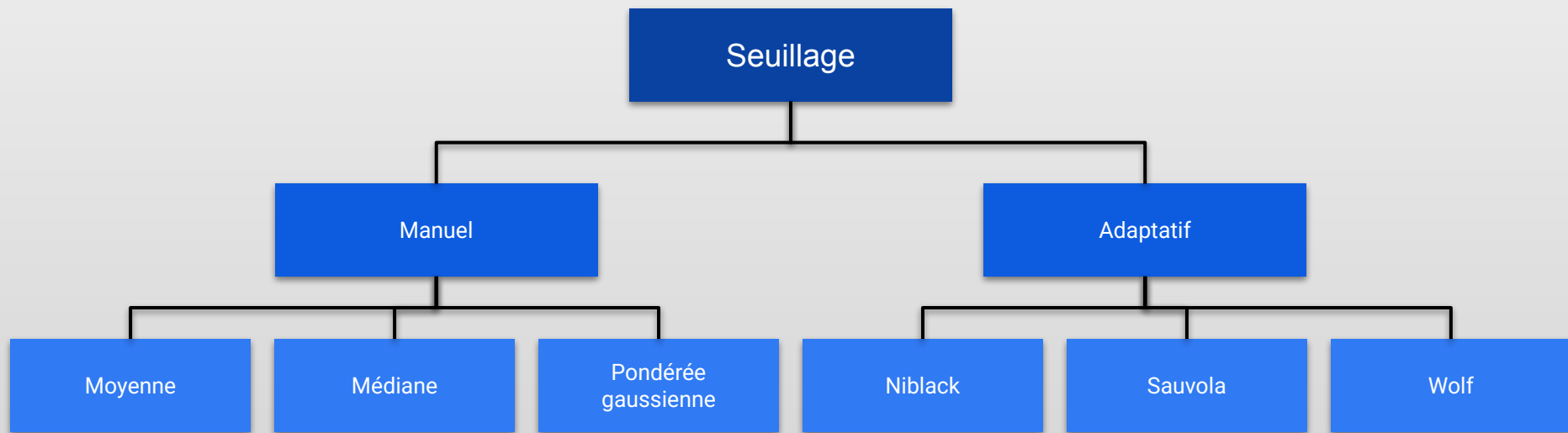
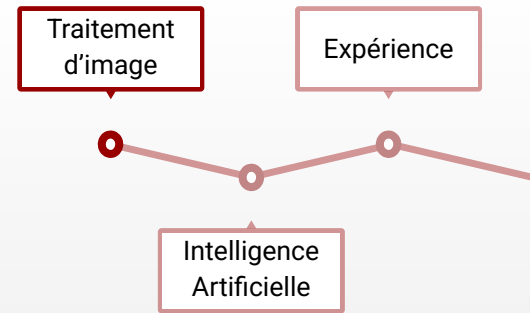
Luma

Image display 'Teste2.bmp' in shades of grey (Luma method)

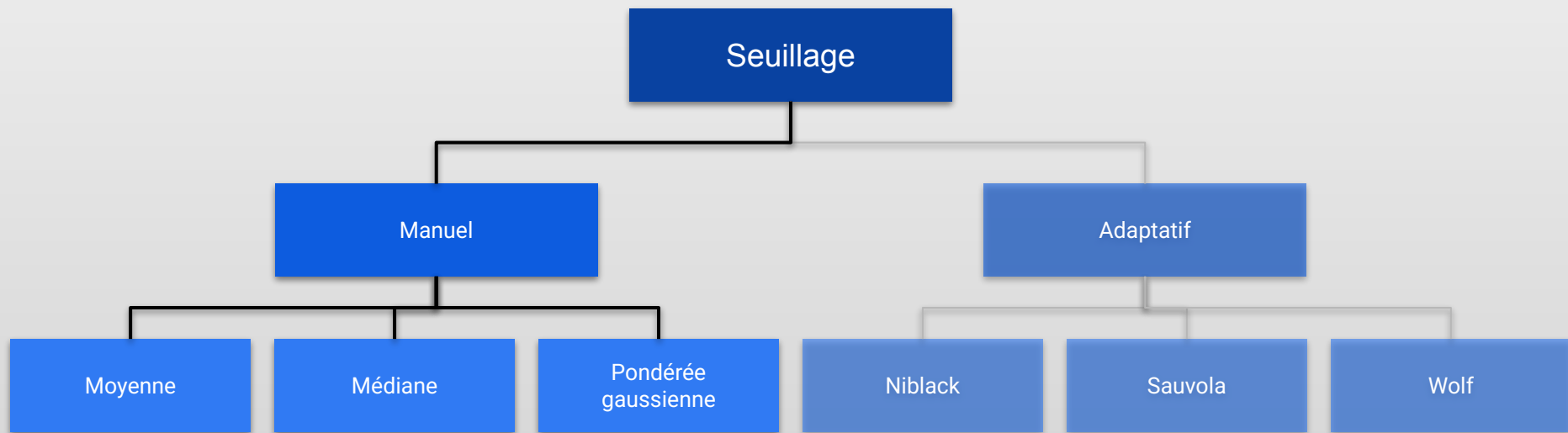
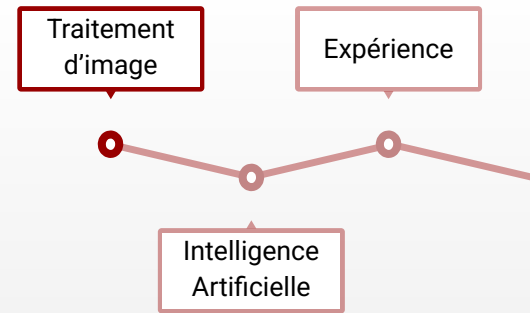
Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité faite à l'image de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "Ils décidèrent de ne transférer leur droit



Traitement d'image (TI)



Traitement d'image (TI)



Traitement d'image (TI)

Pouvoir; soumission, protection de l'individu:
 Chapitre 20: "Semblait être la divinité toute-puissante de
 toutes les traditions qu'il avait revécues." May
 Chapitre XVII: "ils décident de ne transférer leur droit

Pouvoir

Manuel

Moyenne

Segmented Image display 'Teste2.bmp'



(x-n,y-n)			(x-n,y)			(x-n,y+n)
		(x-1,y-1)	(x-1,y)	(x-1,y+1)		
(x,y-n)		(x,y-1)	(x,y)	(x,y+1)		(x,y+n)
		(x+1,y-1)	(x+1,y)	(x+1,y+1)		
(x+n,y-n)			(x+n,y)			(x+n,y+n)

Traitement
d'image

Expérience

Intelligence
Artificielle

Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait révéleres." May
Chapitre XVII: "ils décident de ne transférer leur droit

Pouvoir

Manuel

Moyenne

Médiane

Segmented Image display 'Teste2.bmp'



Segmented Image display 'Teste2.bmp'



Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit

Pouvoir

Manuel

Moyenne

Médiane

Pondérée gaussienne

Segmented Image display 'Teste2.bmp'

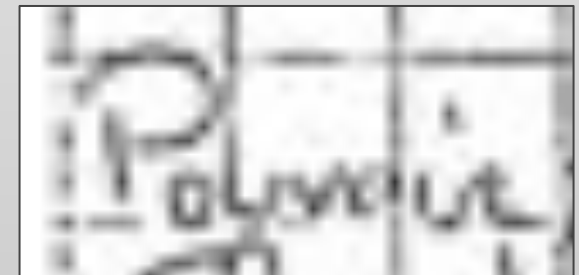


Segmented Image display 'Teste2.bmp'

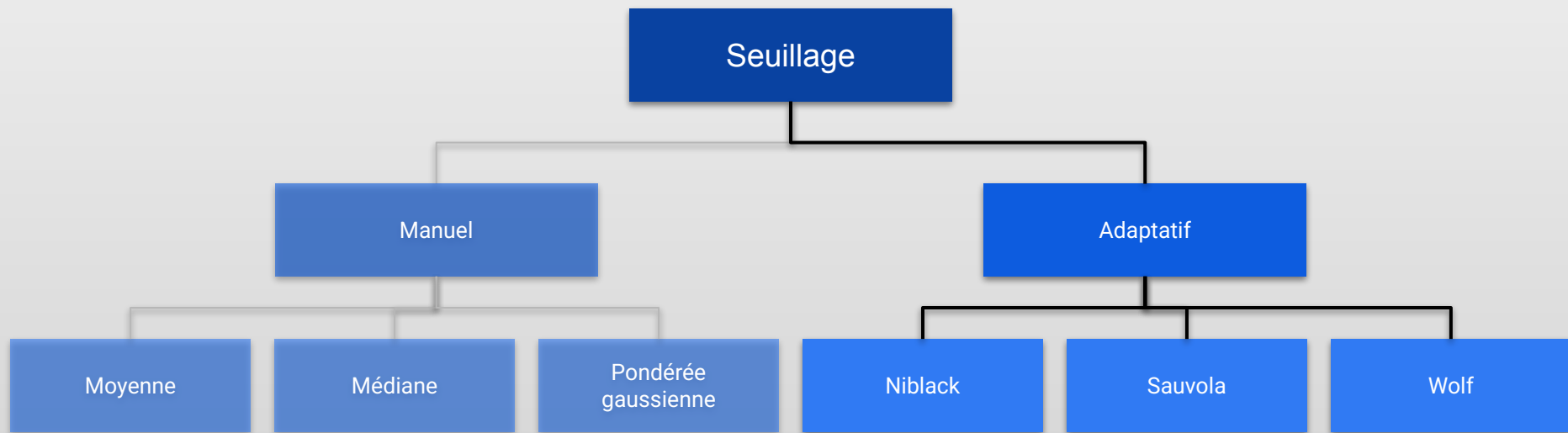
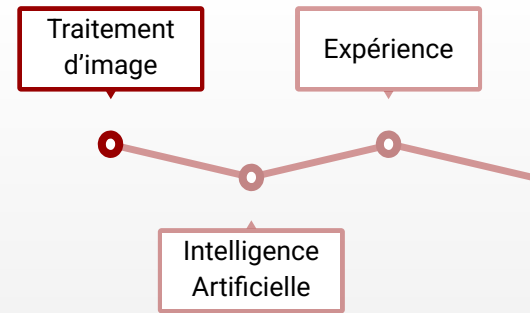


Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit

Pouvoir; soumission, protection de l'individu:
Chapitre 20: "Semblait être la divinité tutélaire de
toutes les traditions qu'il avait revécues." May
Chapitre XVII: "ils décidaient de ne transférer leur droit



Traitement d'image (TI)



Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

Pouvoir : soumission, protection de l'individu :
Chapitre 20 : "Semblait être la divinité faite à l'image de
toutes les traditions qu'il avait sévères." May
Chapitre XVII : "ils décidèrent de ne transférer leur droit

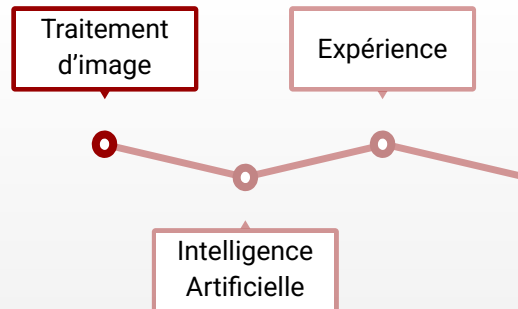
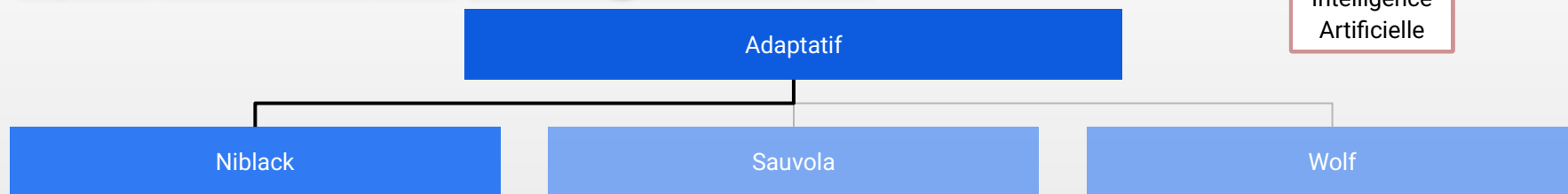
Adaptatif

Niblack

Sauvola

Wolf

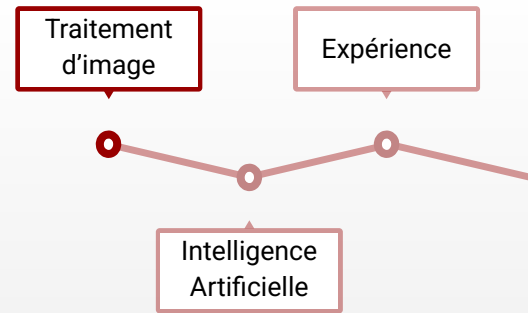
Traitement d'image (TI)



$$T(x, y) = M(x, y) + k \times \sigma(x, y)$$

	k=0.2	k=0.5
n=2	Niblack segmentation	Niblack segmentation
n=8	Niblack segmentation	Niblack segmentation

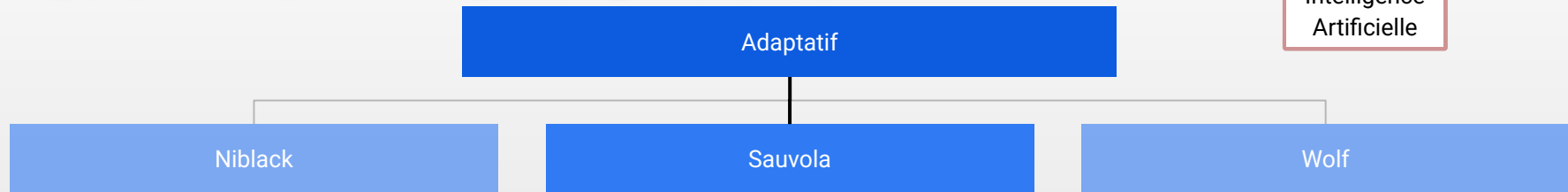
Traitement d'image (TI)



$$T(x, y) = M(x, y) + k \times \sigma(x, y)$$

	k=0.2	k=0.5
n=2		
n=8		

Traitement d'image (TI)



$$T(x, y) = M(x, y) \times \left(1 + k \times \left(\frac{\sigma(x, y)}{R - 1} - 1 \right) \right)$$

	k=0.2	k=0.5
n=2	Sauvola segmentation	Sauvola segmentation
n=8	Sauvola segmentation	Sauvola segmentation

Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

Adaptatif

Niblack

Sauvola

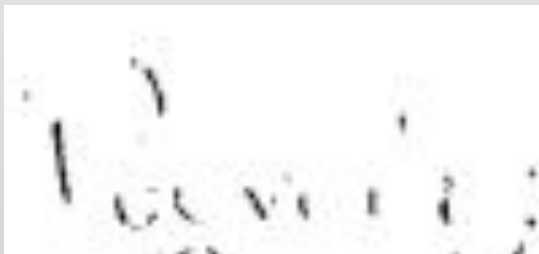
Wolf

$$T(x, y) = M(x, y) \times \left(1 + k \times \left(\frac{\sigma(x, y)}{R - 1} - 1 \right) \right)$$

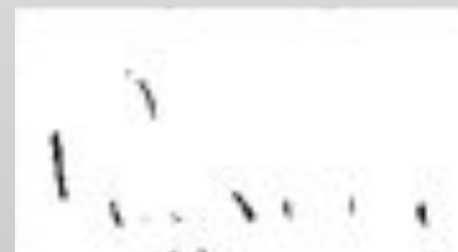
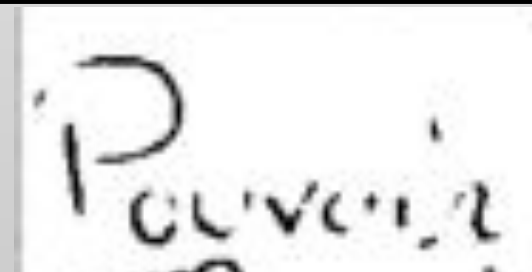
k=0.2

k=0.5

n=2



n=8



Traitement d'image (TI)



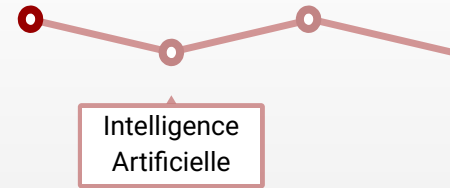
$$T(x, y) = M(x, y) \times (1 - k) + k \times \left(\frac{\sigma(x, y)}{R} \times (Max(I) - I(x, y)) \right)$$

	k=0.2	k=0.5
n=2	Wolf segmentation	Wolf segmentation
n=8	Wolf segmentation	Wolf segmentation

Traitement d'image (TI)

Traitement d'image

Expérience



Adaptatif

Niblack

Sauvola

Wolf

$$T(x, y) = M(x, y) \times (1 - k) + k \times \left(\frac{\sigma(x, y)}{R} \times (Max(I) - I(x, y)) \right)$$

	k=0.2	k=0.5
n=2		
n=8		

Traitement d'image (TI)

Traitement
d'image

Expérience

Intelligence
Artificielle

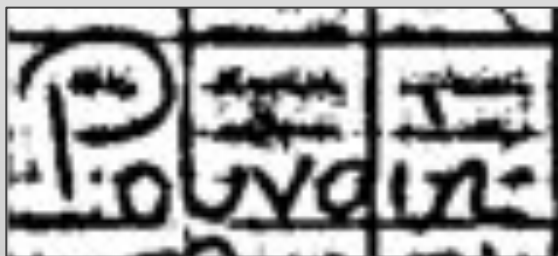
Pouvoir : soumission, protection de l'individu.
Chapitre 10 : "Semblait être la divinité faite à l'aise de
toutes les traditions qu'il avait sévères." May
Chapitre XVII : "ils décidaient de ne transférer leur droit

Adaptatif

Niblack

Niblack segmentation

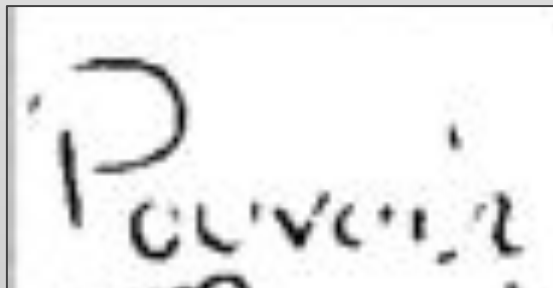
Pouvoir : soumission, protection de l'individu.
Chapitre 10 : "Semblait être la divinité faite à l'aise de
toutes les traditions qu'il avait sévères." May
Chapitre XVII : "ils décidaient de ne transférer leur droit



Sauvola

Sauvola segmentation

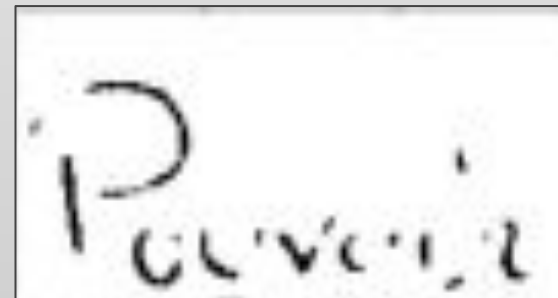
Pouvoir : soumission, protection de l'individu.
Chapitre 10 : "Semblait être la divinité faite à l'aise de
toutes les traditions qu'il avait sévères." May
Chapitre XVII : "ils décidaient de ne transférer leur droit



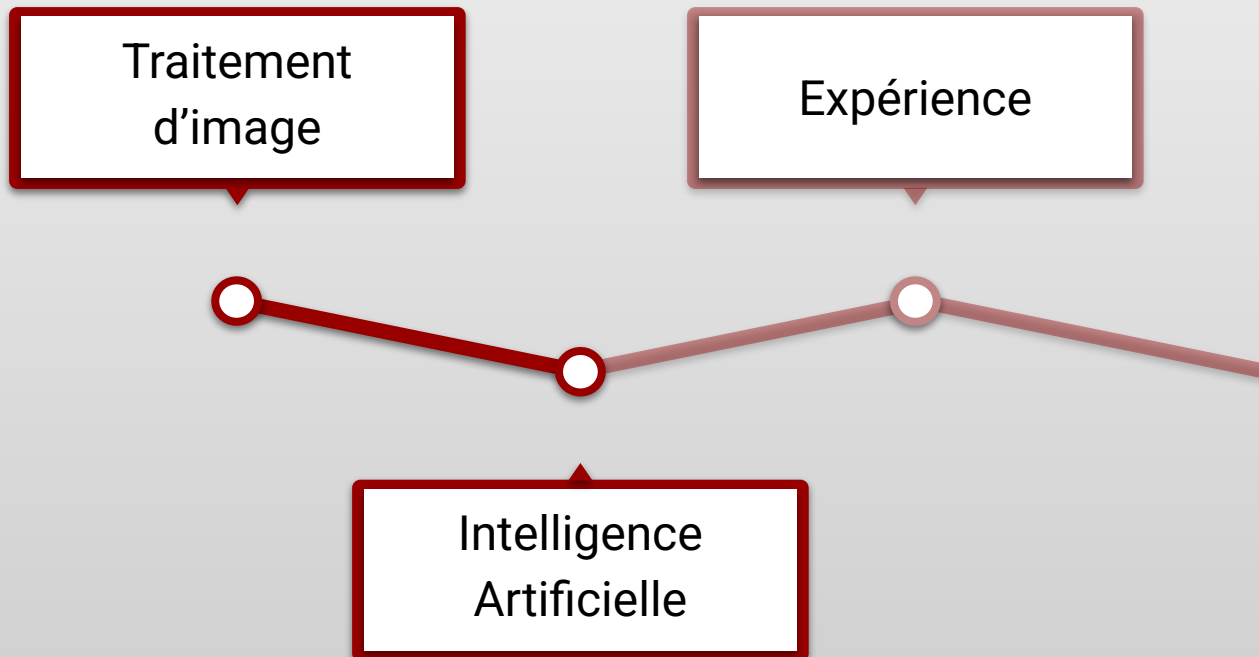
Wolf

Wolf segmentation

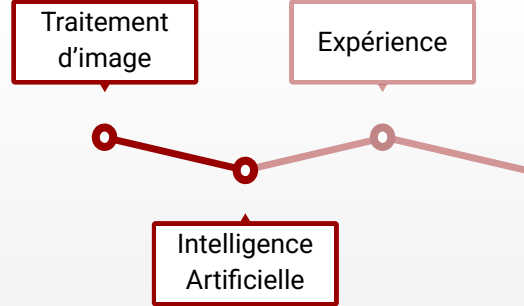
Pouvoir : soumission, protection de l'individu.
Chapitre 10 : "Semblait être la divinité faite à l'aise de
toutes les traditions qu'il avait sévères." May
Chapitre XVII : "ils décidaient de ne transférer leur droit



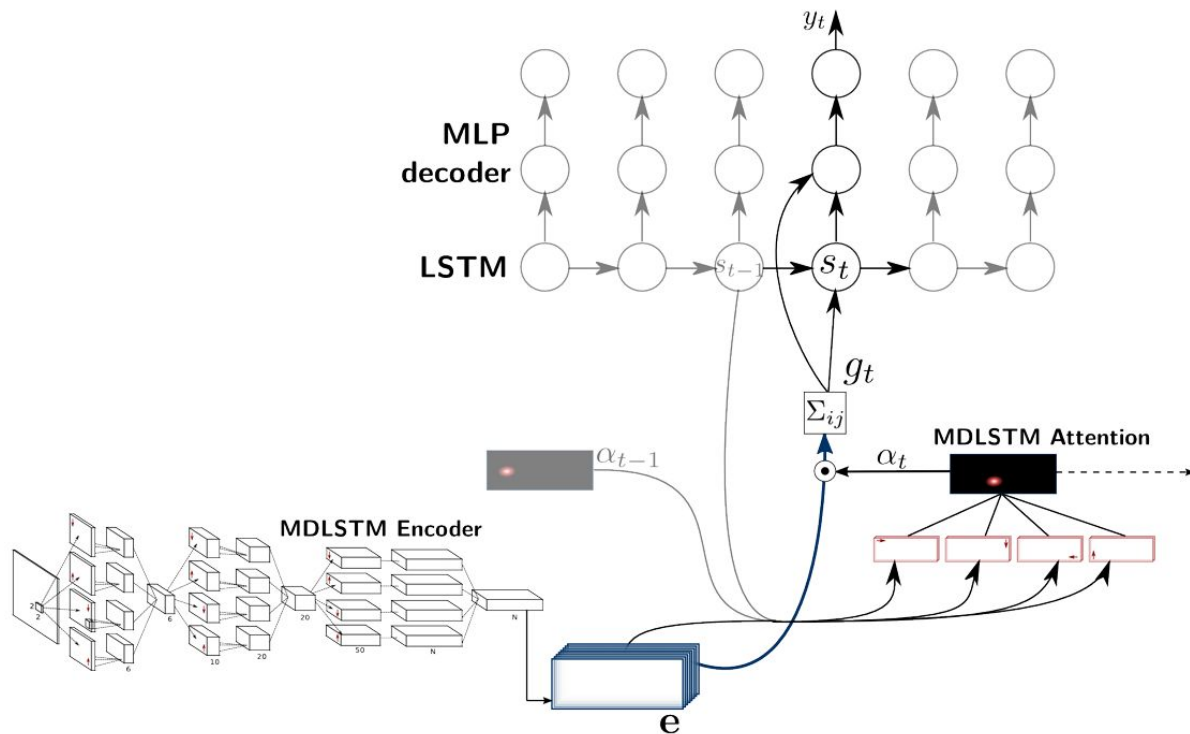
Intelligence artificielle (IA)



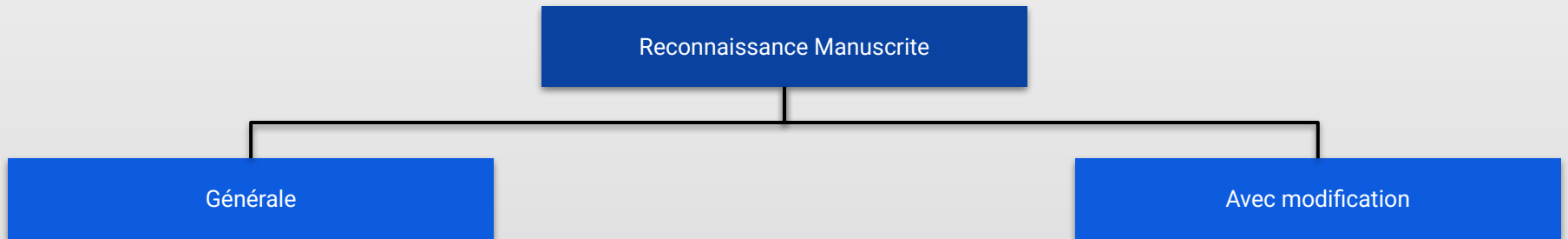
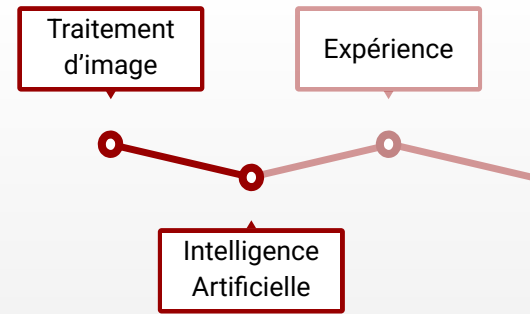
Intelligence artificielle (IA)



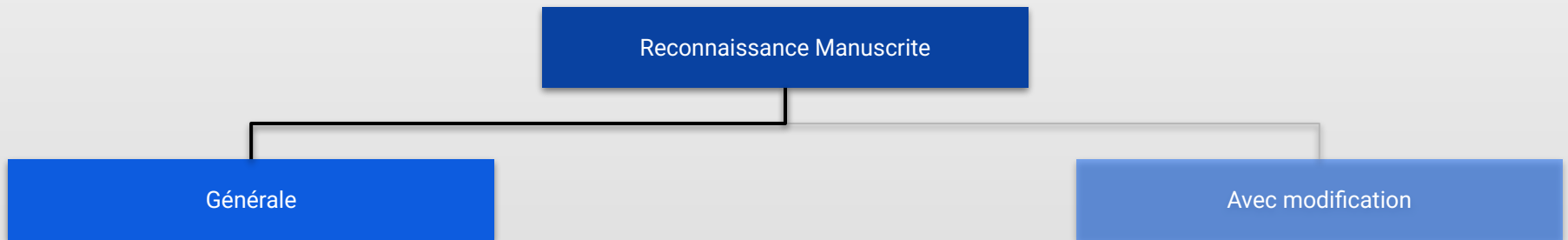
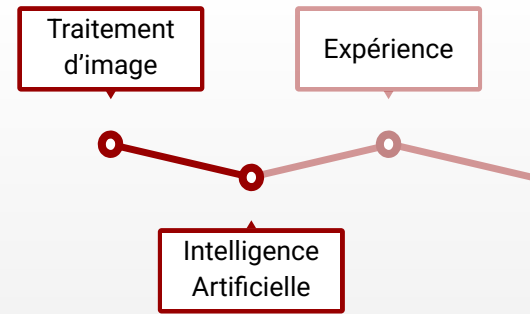
MDLSTM - RNN: Multi-Dimensional Long Short-Term Memory Recurrent Neural Networks



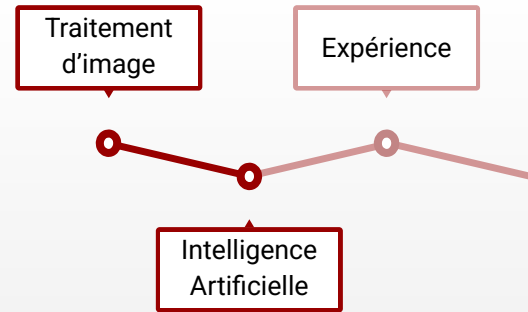
Intelligence artificielle (IA)



Intelligence artificielle (IA)



Intelligence artificielle (IA)

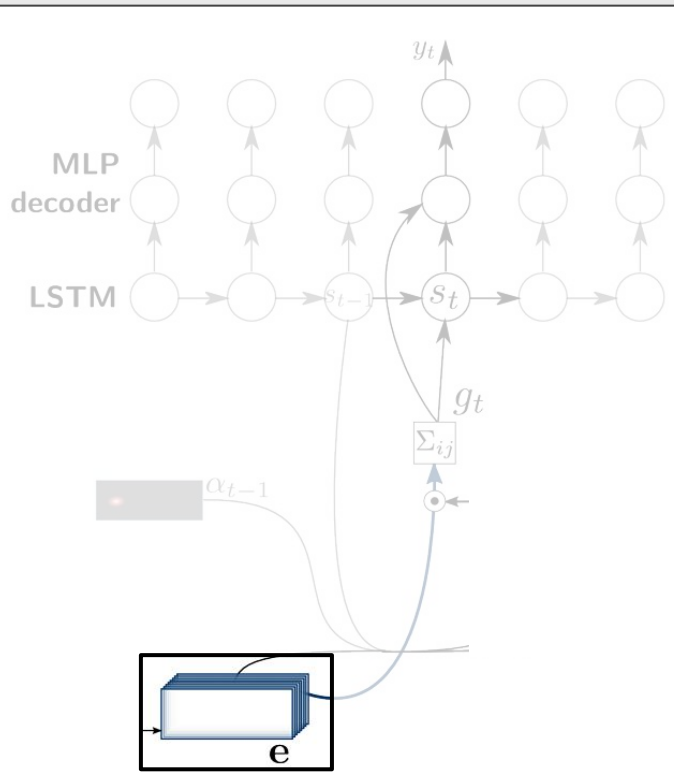


Reconnaissance Manuscrite

Générale

Encodeur:

Avec modification



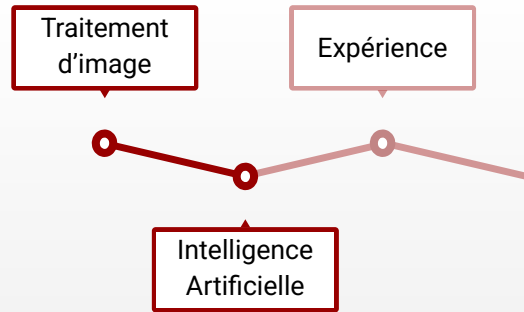
CNN (Convolutional Neural Network):

Couches de convolution

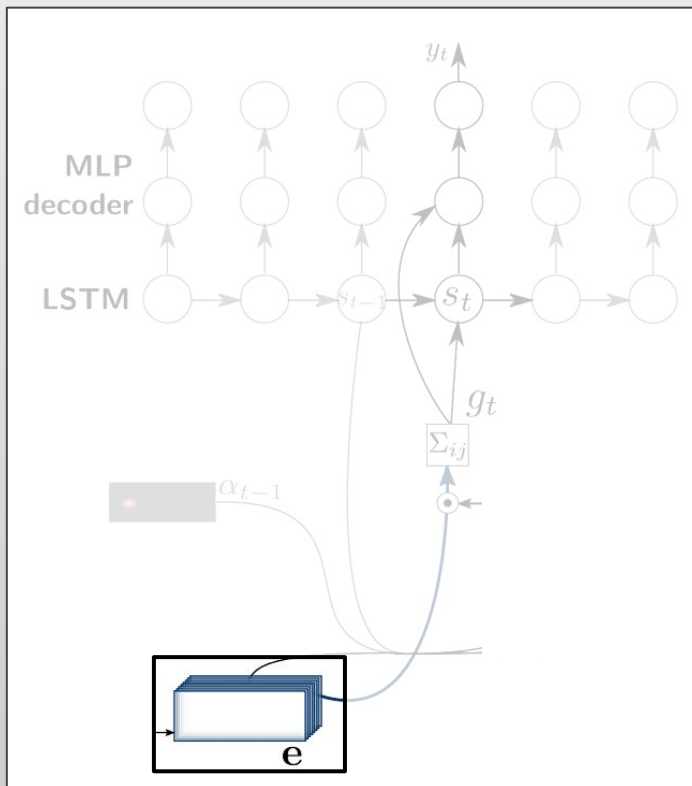
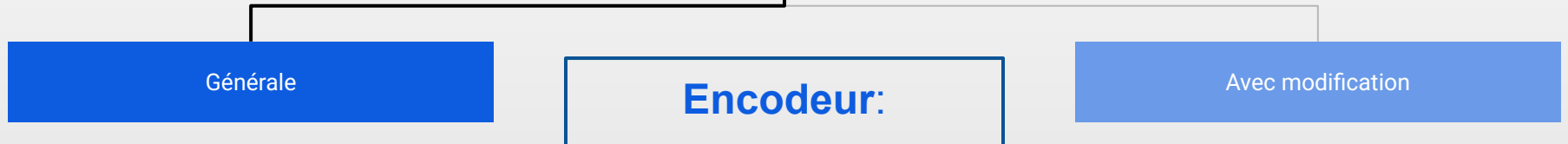
Couches de pooling

~~Couches entièrement connectées~~

Intelligence artificielle (IA)



Reconnaissance Manuscrite



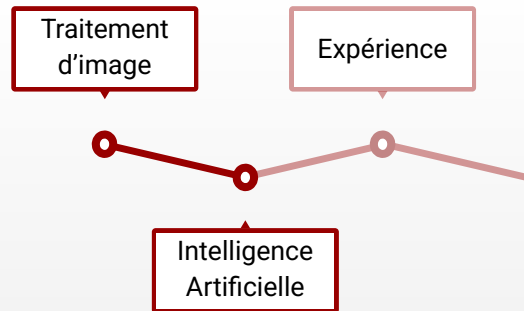
Couches de convolution: Filtre de Sobel

-1	0	1
-2	0	2
-1	0	1

-1	-2	-1
0	0	0
1	2	1

Couches de pooling
Couches entièrement connectées

Intelligence artificielle (IA)

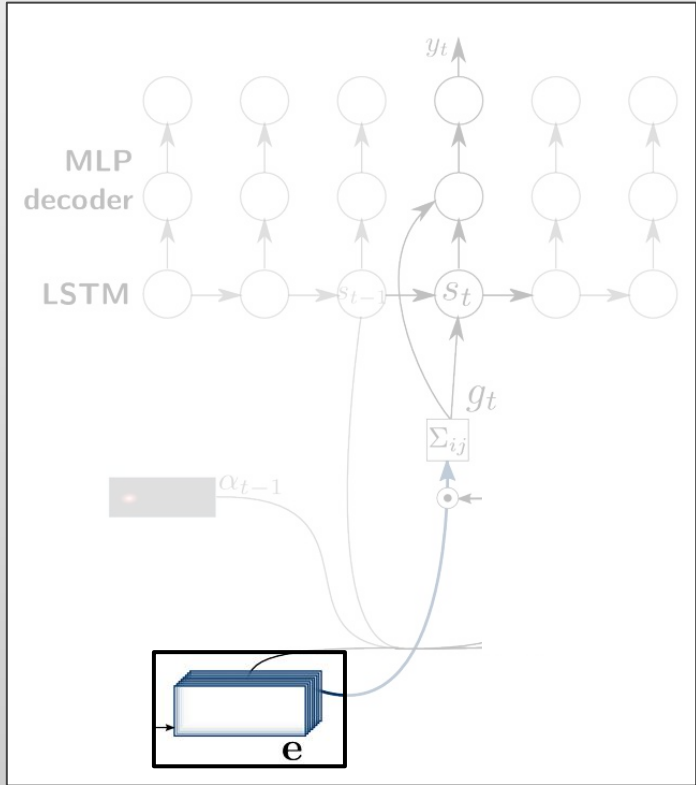


Reconnaissance Manuscrite

Générale

Encodeur:

Avec modification

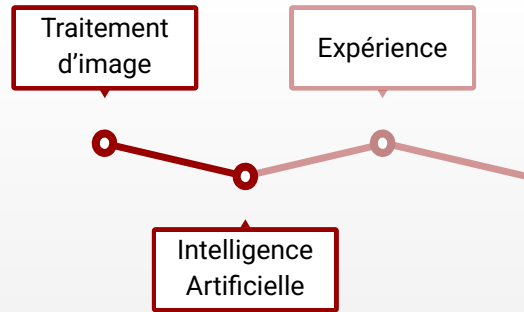


Couches de convolution:
Filtre de Sobel



Couches de pooling
~~Couches entièrement connectées~~

Intelligence artificielle (IA)

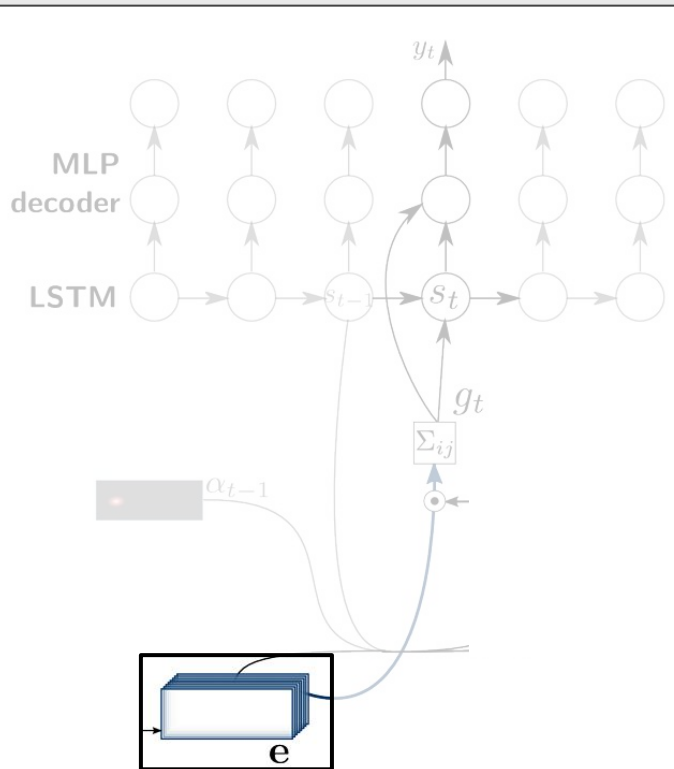


Reconnaissance Manuscrite

Générale

Encodeur:

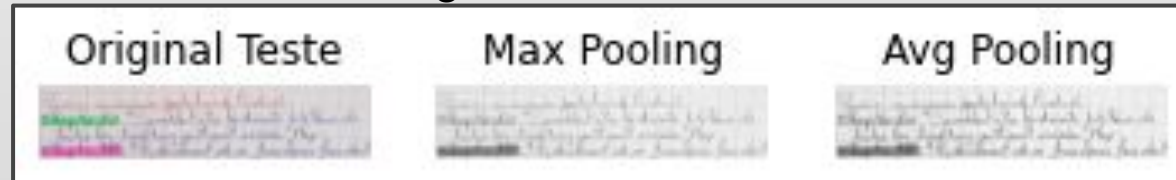
Avec modification



Couches de convolution

Couches de pooling:

Filtre de Pooling:



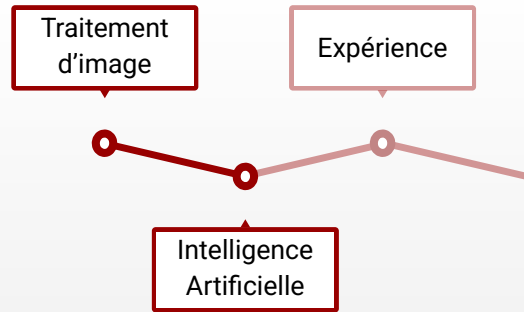
1	4	5
2	3	7
9	8	6

Pooling

9

Couches entièrement connectées

Intelligence artificielle (IA)

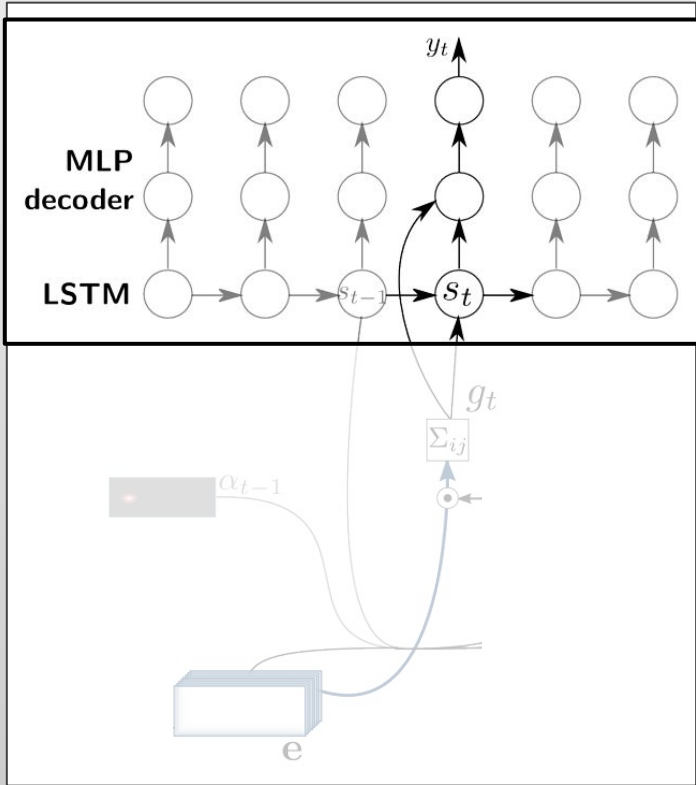


Reconnaissance Manuscrite

Générale

Décodeur:

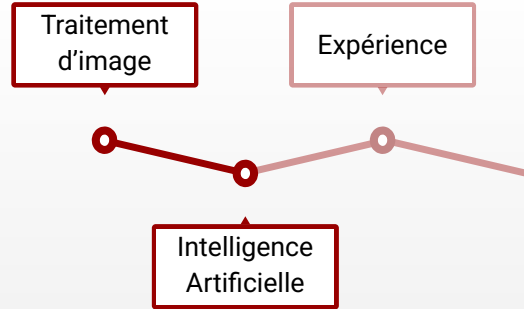
Avec modification



LSTM (Long Short-Term Memory)

Forget Gate
Input Gate
Output Gate

Intelligence artificielle (IA)

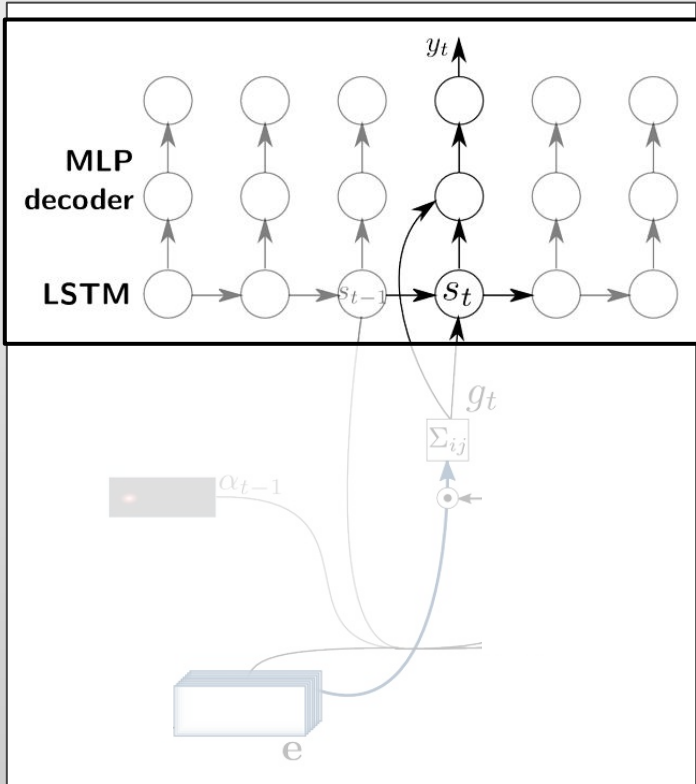


Reconnaissance Manuscrite

Générale

Décodeur:

Avec modification



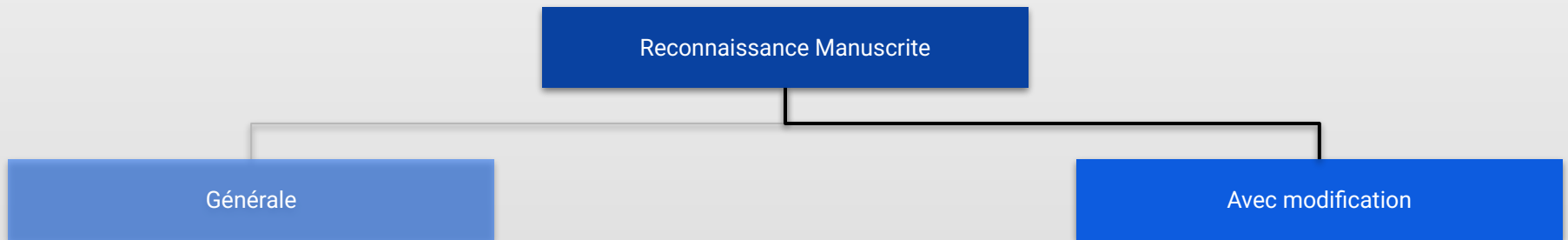
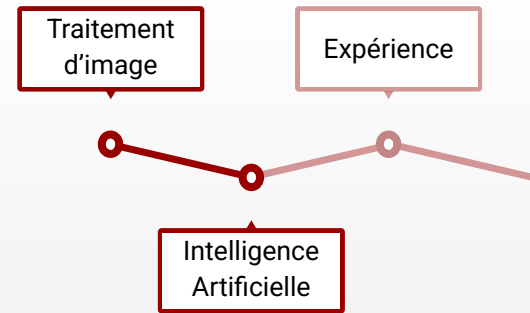
~~LSTM (Long Short Term Memory)~~
GRU (Gated Recurrent Unit)

$$z_t = \sigma(W_z \cdot [x_t, h_{t-1}])$$

$$\tilde{h}_t = \tanh(W_h \cdot [x_t, \sigma(W_r \cdot [x_t, h_{t-1}]) \times h_{t-1}])$$

$$h_t = (1 - z_t) \times h_{t-1} + z_t \times \tilde{h}_t$$

Intelligence artificielle (IA)



Intelligence artificielle (IA)

Traitement
d'image

Expérience



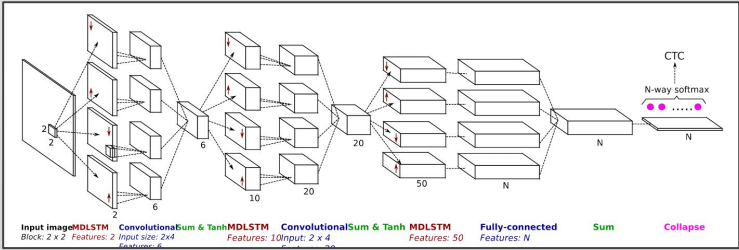
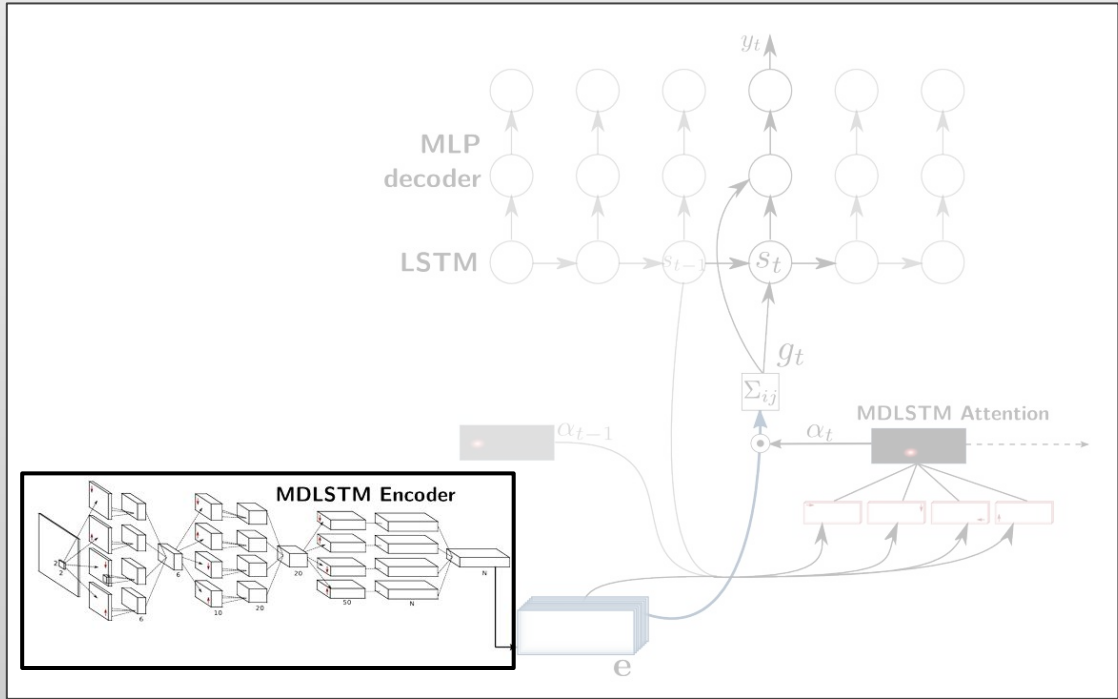
Intelligence
Artificielle

Reconnaissance Manuscrite

Générale

Encodeur MDLSTM:

Avec modification



Intelligence artificielle (IA)

Traitement
d'image

Expérience



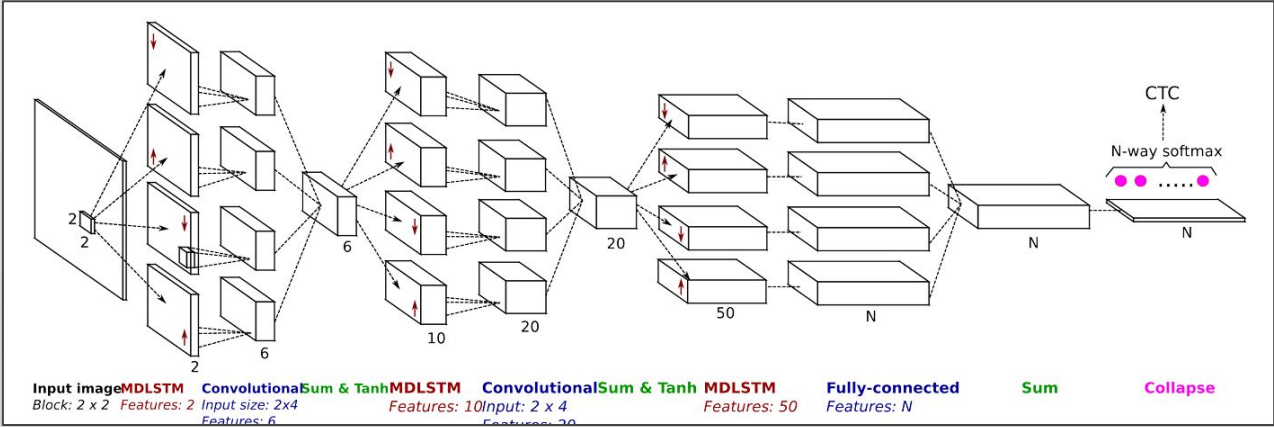
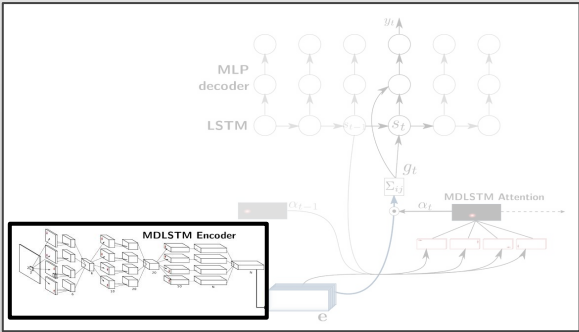
Intelligence
Artificielle

Reconnaissance Manuscrite

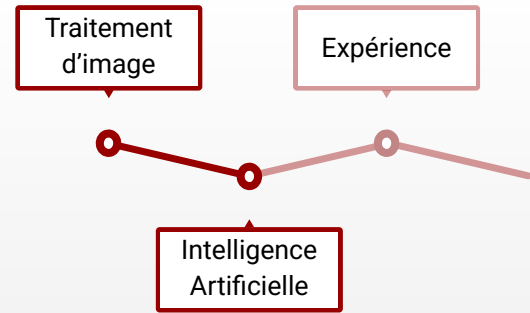
Générale

Encodeur MDLSTM:

Avec modification



Intelligence artificielle (IA)

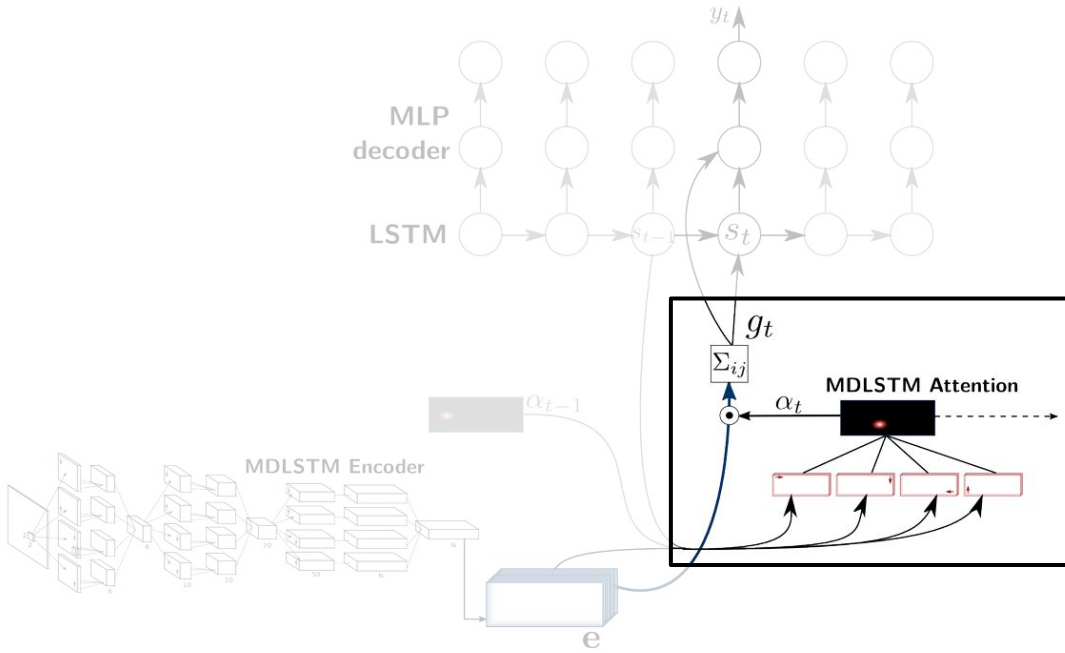


Reconnaissance Manuscript

Générale

Attention:

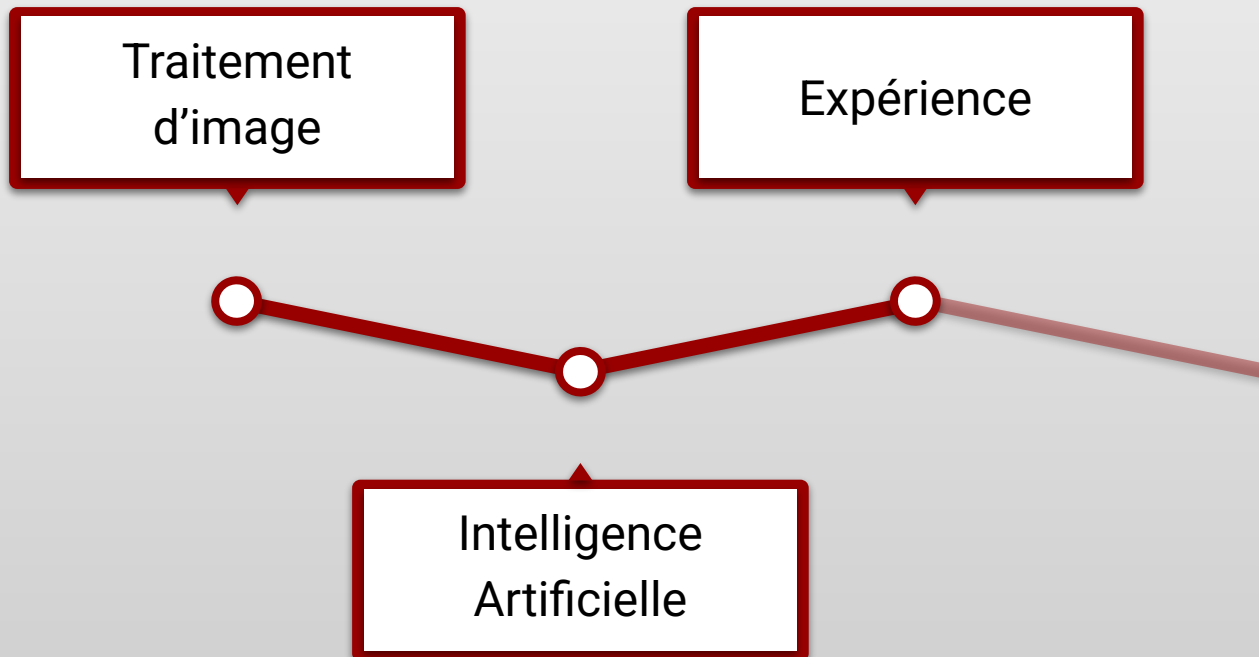
Avec modification



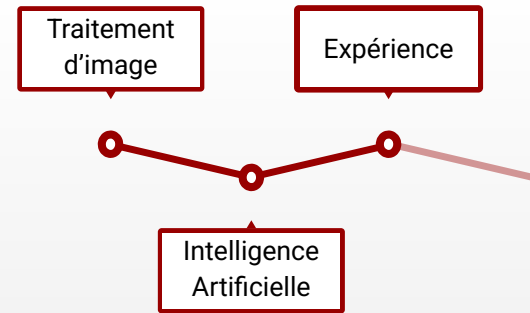
$$\alpha_{(i,j),t} = \frac{e^{z_{(i,j),t}}}{\sum_{i',j'} e^{z_{(i',j'),t}}}$$

$$g_t = \sum_{i,j} \alpha_{(i,j),t} e_{i,j}$$

Expérience



Expérience



Vérification

Comparaison:

'Détection des similarité'	'Detection de simillarité'
----------------------------	----------------------------

Renvoie:

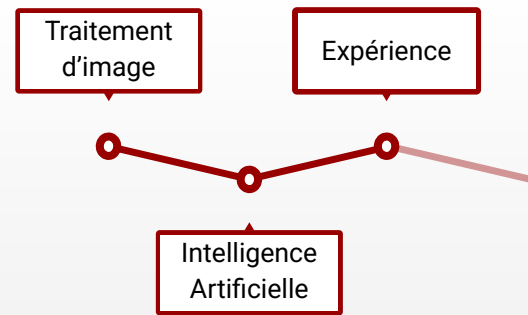
Similarity : 91.67%

Differences :

D[é≠e]tection de[+s] simil[-l]arité

Légende : [a≠b] = wrong letter, [+a] = missing letter, [-b] = excess letter

Expérience

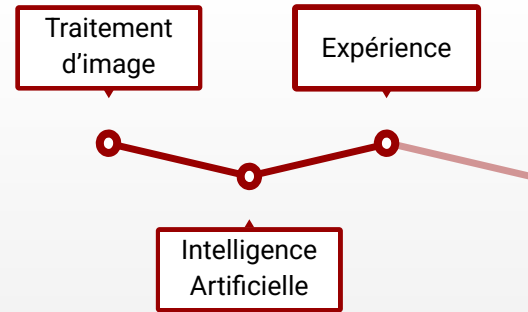


$k=0.2$

$n=9$

Pouvoir, soumission, protection de l'individu
Chapitre 20: "Semblait être la divinité toute-puissante de
toutes les traditions qu'il avait revécues." May
Chapitre XVIII: "ils décidaient de ne transférer leur droit

Expérience



True: hello | Pred: text

hello

True: hello | Pred: world

hello

True: hello | Pred: hello

hello

True: hello | Pred: hello

hello

True: hello | Pred: world

hello

True: hello | Pred: text

hello

True: hello | Pred: hello

hello

True: hello | Pred: text

hello

True: hello | Pred: hello

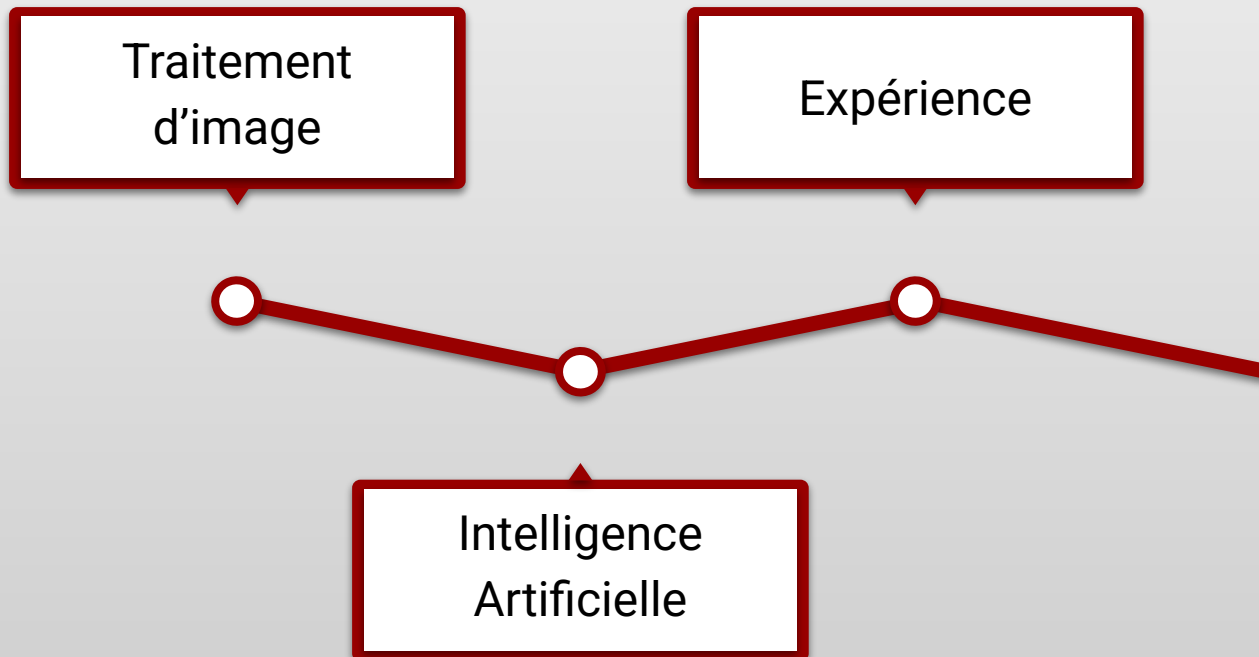
hello

True: hello | Pred: text

hello

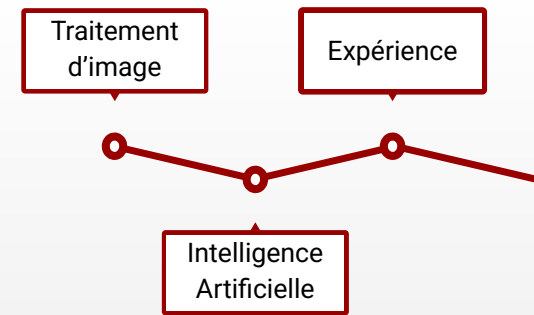
40%

Conclusion et discussion



Conclusion et discussion

- Réduction du temps de calcul
- Utilisation des seuillages adaptatif
- Reconnaissance avec Vanilla Beam Search
- Pré-Seuillage avec la couleur



Fin

Annexe I: Bibliothèques

- `from matplotlib import pyplot as plt`
- `import numpy as np`
- `from PIL import Image`

Intelligence artificielle

- `import os`
- Pytorch:
 - `import torch`
 - `import torch.nn as nn`
 - `import torch.nn.functional as F`
 - `import matplotlib.pyplot as plt`
 - `from torch.utils.data import Dataset, DataLoader`

Annexe 2: Source

Traitement d'image:

- Digital Image Processing
- Mesures de similarité entre images dans le cadre des problèmes inverses
- Adaptive Document Binarization
- A Threshold Selection method from gray-Level histograms

Intelligence artificielle:

- Scan, Attend and Read: End-to-End Handwritten Paragraph Recognition with MDLSTM Attention
- Wikipedia

Annexe 3: Convert jpg to bmp (Tl)

```
def convert_jpg_to_bmp(name):  
    """  
    Convert png images to bmp.  
  
    Input:  
        name (str): name of the image we want  
  
    Output:  
        array: reading each pixel on bmp image.  
    """  
    try:  
        Im = plt.imread(name + '2.bmp')  
        print(f'A file already exists {name}2.bmp')  
        return Im  
    except FileNotFoundError:  
        print("A file does not already exists {nom}2.bmp")  
        img = Image.open(name + '.jpg') # Open image  
        img.save(name + '2.bmp', "BMP") # Save image converted to BMP  
        return plt.imread(name + '2.bmp')
```

Annexe 4: Simple average (T1)

```
def shades_grey(im):  
    """  
    Change a coloured image ('RGB') to black and white.  
  
    Input:  
        im (str): corresponds to reading the image.  
  
    Output:  
        array: greyscale image.  
    """  
    im1 = convert_jpg_to_bmp(im)  
    carac = im1.shape # Knowing the image size  
    re = im1.copy()  
    for i in range(carac[0]): # Browse the lines of pixels in the image  
        for j in range(carac[1]): # Columns of pixels in the image  
            pix = im1[i][j] # Value of the pixel's three colours  
            grey = (pix[0]+pix[1]+pix[2])/3 # The grey value  
            re[i][j] = [grey, grey, grey] # Replaces the pixel  
    return re
```

Annexe 5: Maximum brightness (Tl)

```
def shades_grey(im):  
    """  
    Change a coloured image ('RGB') to black and white.  
  
    Input:  
        im (str): corresponds to reading the image.  
  
    Output:  
        array: greyscale image.  
    """  
    im1 = convert_jpg_to_bmp(im)  
    carac = im1.shape # Knowing the image size  
    re = im1.copy()  
    for i in range(carac[0]): # Browse the lines of pixels in the image  
        for j in range(carac[1]): # Columns of pixels in the image  
            pix = im1[i][j] # Value of the pixel's three colours  
            grey = max(pix) # The grey value  
            re[i][j] = [grey, grey, grey] # Replaces the pixel  
    return re
```

Annexe 6: Luma (Tl)

```
def shades_grey(im):  
    """  
    Change a coloured image ('RGB') to black and white.  
  
    Input:  
        im (str): corresponds to reading the image.  
  
    Output:  
        array: greyscale image.  
    """  
    im1 = convert_jpg_to_bmp(im)  
    carac = im1.shape # Knowing the image size  
    re = im1.copy()  
    for i in range(carac[0]): # Browse the lines of pixels in the image  
        for j in range(carac[1]): # Columns of pixels in the image  
            pix = im1[i][j] # Value of the pixel's three colours  
            grey = int(sum([0.176, 0.813, 0.011]*pix)) # The grey value  
            re[i][j] = [grey, grey, grey] # Replaces the pixel  
    return re
```

Annexe 7: Convert integer and binary (T1)

```
def convert_to_binary(image):  
    """  
    Convert binary image to integer.  
  
    Input:  
        binary_image (array):  
  
    Output:  
        integer_image (array):  
    """  
    binary_image = []  
    for row in image:  
        binary_row = []  
        for pixel in row:  
            binary_pixel = format(pixel[0], '08b')  
            binary_row.append([int(bit) for bit in  
binary_pixel])  
        binary_image.append(binary_row)  
    return binary_image
```

```
def convert_to_integer(binary_image):  
    """  
    Convert binary image to integer.  
  
    Input:  
        binary_image (array):  
  
    Output:  
        integer_image (array):  
    """  
    integer_image = []  
    for row in binary_image:  
        integer_row = []  
        for pixel in row:  
            pixel_value = sum(  
                [pixel[i] * 2 ** (7 - i)) for i in  
range(len(pixel))])  
            integer_row.append([pixel_value for k  
in range(3)])  
        integer_image.append(integer_row)  
    return np.array(integer_image)
```

Annexe 8: Bit planes slicing (TI)

```
def bit_plane_slicing(image, num_bits_to_keep=4):
    """
    Reduce.

    Input:
        image (array)
        num_bits_to_keep (int)
    Output:
        reduced_image (array)
    """
    reduced_image = []
    binary_image =
convert_to_binary(shades_grey(image))
    for row in binary_image:
        reduced_row = []
        for pixel in row:
            reduced_pixel = pixel[-num_bits_to_keep:]
            reduced_row.append(reduced_pixel)
        reduced_image.append(reduced_row)
    reduced_image = convert_to_integer(reduced_image)
    return reduced_image

def bit_plane_slicing(image):
    """
    Perform bit-plane slicing on a grayscale image.

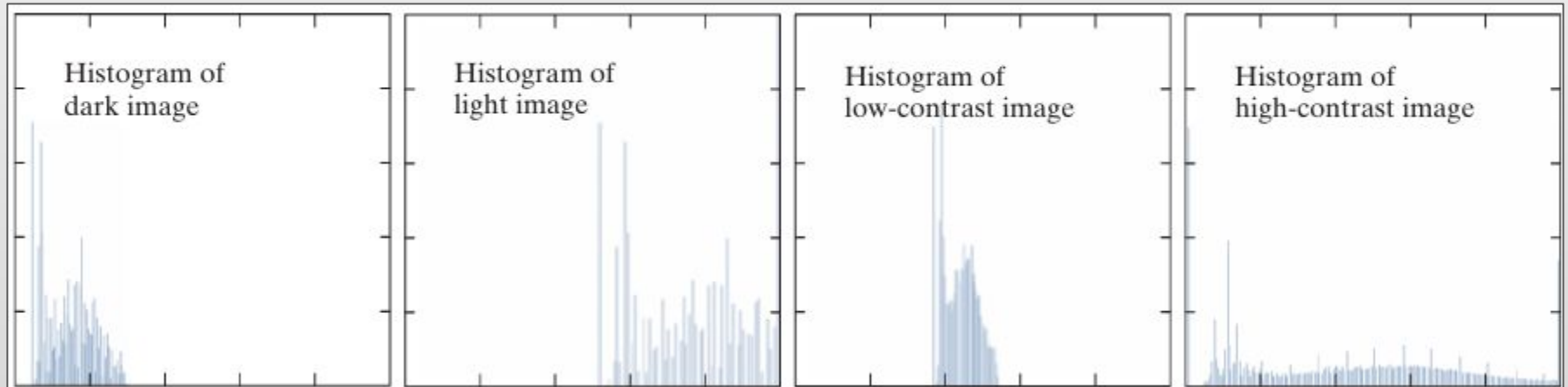
    Input:
        image (str): name of the image file.

    Output:
        list of images corresponding to each bit-plane.
    """
    H_reduced_image = []
    binary_image = convert_to_binary(shades_grey(image))
    for num_bits_to_keep in range(9):
        reduced_image = []
        for row in binary_image:
            reduced_row = []
            for pixel in row:
                reduced_pixel = pixel[-num_bits_to_keep:]
                if (num_bits_to_keep > 0) else [0] * 8
                reduced_row.append(reduced_pixel)
            reduced_image.append(reduced_row)
    H_reduced_image.append(convert_to_integer(reduced_image))
    print(f"Check coat - {num_bits_to_keep}")
    return H_reduced_image
```


Annexe 9: Comparison of shades of grey (T1)

```
def histogramme (im) :  
    """  
    Display histogramme.  
  
    Input:  
        im (array): greyscale image.  
  
    Output:  
        list: Number of Pixels  
    """  
    L = [0]*256  
    for i in im:  
        for j in i:  
            L[j[0]] += 1  
    return L
```

Annexe 10: Diagramme histogramme (T1)



Annexe II: Weight of the Gaussian formula (T1)

```
r = np.linspace(0, 5, 50)
w = np.exp(-(r)**2 / 2)
z = [0 for k in range(len(w))]
```

Annexe 12: Gaussian (T1)

```
def matrix_average(matrix):  
    """  
    Compute the average of a matrix.
```

Input:

matrix (np.array): 2D array.

Output:

float: Average value.

```
    """  
    return np.mean(matrix)
```

```
def create_gaussian_kernel(n):  
    """  
    Create a 1D Gaussian kernel.
```

Input:

n (int): Radius of the Gaussian kernel.

Output:

np.array: 1D Gaussian kernel.

```
    """  
    x = np.arange(-n, n+1)  
    G = np.exp(-(x**2) / (2 * n**2))  
    G /= G.sum() # Normalization  
    return G
```

```
def convolve_1d(image, k, axis=0):  
    """  
    Perform 1D convolution on an image along the specified  
    axis.
```

Input:

image (np.array): 2D grayscale image.

k (np.array): 1D convolution kernel.

axis (int): Axis along which to apply the
convolution (0 = vertical, 1 = horizontal).

Output:

np.array: Convolved image.

```
    """  
    from scipy.ndimage import convolve1d  
    return convolve1d(image, k, axis=axis, mode='reflect')
```

```
def Gaussian(n, K, image_path):  
    """  
    Apply a Gaussian filter using separable convolution.
```

Input:

n (int): Radius of the Gaussian kernel.

K (float): Scaling constant.

image_path (str): Path to the image.

Output:

np.array: Filtered image.

```
    """  
    im1 = Luma(image_path)  
    G = create_gaussian_kernel(n)  
    temp = convolve_1d(im1, G, axis=1)  
    result = convolve_1d(temp, G, axis=0)  
    return result
```

Annexe 13: Local average (T1)

```
def local_average(im, n, C):  
    """  
    Segment the image.  
  
    Input:  
        im (str): corresponds to reading the image.  
        n (int): close neighbour  
        C (int): Adjustment constant  
  
    Output:  
        array: Segmented image.  
    """  
    im1 = shades_grey(im)  
    carac = im1.shape # Knowing the image size  
    re = im1.copy()  
    for i in range(n, carac[0]-n): # Browse the lines  
        for j in range(n, carac[1]-n): # Browse the columns  
            T = im1[i-n:i+n+1][j-n:j+n+1].mean() - C  
            if im1[i, j].mean() >= T:  
                re[i][j] = [255, 255, 255]  
            else:  
                re[i][j] = [0, 0, 0]  
    return re
```

Annexe 14: Local median (T1)

```
def local_median(im, n):  
    """  
    Applies a local median threshold to binarize the image.  
  
    Input:  
        im (str): filename (without extension) of the image.  
        n (int): half-size of the local window.  
  
    Output:  
        array: Binarized image using local median.  
    """  
    im1 = shades_grey(im)  
    carac = im1.shape  
    re = im1.copy()  
    for i in range(n, carac[0] - n):  
        for j in range(n, carac[1] - n):  
            L = im1[i - n:i + n + 1, j - n:j + n + 1, 0]  
            median = np.median(L)  
            if im1[i][j][0] >= median:  
                re[i][j] = [255, 255, 255]  
            else:  
                re[i][j] = [0, 0, 0]  
    return re
```

Annexe 15: Niblack (T1)

```
def Niblack(im, n, k):
    im1 = shades_grey(im)
    carac = im1.shape
    re = im1.copy()

    for i in range(n, carac[0] - n):
        for j in range(n, carac[1] - n):
            L = im1[i-n:i+n+1, j-n:j+n+1, 0] # grayscale channel
            M = local_average(L)
            S = local_variance(L, M)
            T = M + k * np.sqrt(S)
            if im1[i][j][0] >= T:
                re[i][j] = [255, 255, 255]
            else:
                re[i][j] = [0, 0, 0]

    return re
```

Annexe 16: Sauvola (TI)

```
def Sauvola(im, n, k, R=128):
    im1 = shades_grey(im)
    carac = im1.shape
    re = im1.copy()

    for i in range(n, carac[0] - n):
        for j in range(n, carac[1] - n):
            L = im1[i-n:i+n+1, j-n:j+n+1, 0] # grayscale channel
            M = local_average(L)
            S = local_variance(L, M)
            T = M * (1 - k * (np.sqrt(S) / (R-1) - 1))
            if im1[i][j][0] >= T:
                re[i][j] = [255, 255, 255]
            else:
                re[i][j] = [0, 0, 0]

    return re
```


Annexe 17: Wolf (TI)

```
def Wolf(im, n, k, R=128):
    im1 = shades_grey(im)
    carac = im1.shape
    re = im1.copy()
    M_values = []
    for i in range(n, carac[0] - n):
        for j in range(n, carac[1] - n):
            L = im1[i - n:i + n + 1, j - n:j + n + 1, 0]
            M = local_average(L)
            M_values.append(M)
    Mmax = max(M_values)
    for i in range(n, carac[0] - n):
        for j in range(n, carac[1] - n):
            L = im1[i - n:i + n + 1, j - n:j + n + 1, 0]
            M = local_average(L)
            S = np.sqrt(local_variance(L, M))
            T = M - k * (Mmax - M) * (1 - S / R)

            if im1[i][j][0] >= T:
                re[i][j] = [255, 255, 255]
            else:
                re[i][j] = [0, 0, 0]
    return re
```

Annexe 18: Encodeur MDLSTM (IA)

```
class EncoderCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2),
            nn.Conv2d(64, 128, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )

    def forward(self, x):
        features = self.conv(x) # (B, C, H, W)
        B, C, H, W = features.shape
        return features.view(B, C, -1).permute(0, 2, 1)
```

Annexe 19: Mécanisme d'attention (IA)

```
class Attention(nn.Module):
    def __init__(self, encoder_dim, decoder_dim):
        super().__init__()
        self.attn = nn.Linear(encoder_dim + decoder_dim, 128)
        self.v = nn.Linear(128, 1, bias=False)

    def forward(self, hidden, encoder_outputs):
        B, T, _ = encoder_outputs.shape
        hidden = hidden.unsqueeze(1).repeat(1, T, 1)
        energy = torch.tanh(self.attn(torch.cat((hidden, encoder_outputs), dim=2)))
        attn = self.v(energy).squeeze(2)
        return F.softmax(attn, dim=1)
```

Annexe 20: Décodeur (IA)

```
class DecoderRNN(nn.Module):
    def __init__(self, output_size, encoder_dim=128, hidden_dim=128):
        super().__init__()
        self.hidden_dim = hidden_dim
        self.embedding = nn.Embedding(output_size, hidden_dim)
        self.attention = Attention(encoder_dim, hidden_dim)
        self.rnn = nn.GRU(encoder_dim + hidden_dim, hidden_dim,
batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_size)

    def forward(self, input_step, hidden, encoder_outputs):
        embedded = self.embedding(input_step)
        attn_weights = self.attention(hidden.squeeze(0), encoder_outputs)
        context = torch.bmm(attn_weights.unsqueeze(1), encoder_outputs)
        rnn_input = torch.cat((embedded, context), dim=2)
        output, hidden = self.rnn(rnn_input, hidden)
        output = self.fc(output.squeeze(1))
        return output, hidden, attn_weights
```

Annexe 21: Générateur de texte (1A)

```
def generate_synthetic_image(text="hello", font_size=24, font_path=None):  
    img = Image.new("L", (IMG_WIDTH, IMG_HEIGHT), color=255)  
    draw = ImageDraw.Draw(img)  
    try:  
        font = ImageFont.truetype(font_path or  
"/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf", font_size)  
    except:  
        font = ImageFont.load_default()  
    draw.text((5, 5), text, font=font, fill=0)  
    return img
```

Annexe 22: Sobel (T1)

```
def extract_gray(image_rgb):  
    """  
    Extract grayscale channel from RGB image.  
  
    Input:  
        image_rgb (np.array): RGB image with  
        identical channels.  
    Output:  
        np.array: 2D grayscale image.  
    """  
    return image_rgb[:, :, 0]
```

```
def sobel_filter(im):  
    """  
    Apply Sobel filter to a grayscale image using manual  
    convolution.  
  
    Input:  
        im (str): image name (without extension).  
    Output:  
        np.array: Magnitude of gradient.  
    """  
    im1 = Luma(im)  
    gray = extract_gray(im1)  
    height, width = gray.shape  
    sobel_x = np.array([[ -1,  0,  1],  
                        [ -2,  0,  2],  
                        [ -1,  0,  1]])  
  
    sobel_y = np.array([[ -1, -2, -1],  
                        [  0,  0,  0],  
                        [  1,  2,  1]])  
  
    grad_x = np.zeros_like(gray, dtype=np.float32)  
    grad_y = np.zeros_like(gray, dtype=np.float32)  
    for i in range(1, height-1):  
        for j in range(1, width-1):  
            region = gray[i-1:i+2, j-1:j+2]  
            gx = np.sum(region * sobel_x)  
            gy = np.sum(region * sobel_y)  
            grad_x[i, j] = gx  
            grad_y[i, j] = gy  
    magnitude = np.sqrt(grad_x**2 + grad_y**2)  
    magnitude = (magnitude / magnitude.max()) * 255  
    return magnitude.astype(np.uint8)
```

Annexe 23: Pooling (T1)

```
def extract_gray(image_rgb):  
    """  
    Extract grayscale channel from RGB image.  
  
    Input:  
        image_rgb (np.array): RGB image with  
        identical channels.  
    Output:  
        np.array: 2D grayscale image.  
    """  
    return image_rgb[:, :, 0]
```

```
def sobel_filter(im):  
    """  
    Apply Sobel filter to a grayscale image using manual  
    convolution.  
  
    Input:  
        im (str): image name (without extension).  
    Output:  
        np.array: Magnitude of gradient.  
    """  
    im1 = Luma(im)  
    gray = extract_gray(im1)  
    height, width = gray.shape  
    sobel_x = np.array([[ -1,  0,  1],  
                        [ -2,  0,  2],  
                        [ -1,  0,  1]])  
  
    sobel_y = np.array([[ -1, -2, -1],  
                        [  0,  0,  0],  
                        [  1,  2,  1]])  
  
    grad_x = np.zeros_like(gray, dtype=np.float32)  
    grad_y = np.zeros_like(gray, dtype=np.float32)  
    for i in range(1, height-1):  
        for j in range(1, width-1):  
            region = gray[i-1:i+2, j-1:j+2]  
            gx = np.sum(region * sobel_x)  
            gy = np.sum(region * sobel_y)  
            grad_x[i, j] = gx  
            grad_y[i, j] = gy  
    magnitude = np.sqrt(grad_x**2 + grad_y**2)  
    magnitude = (magnitude / magnitude.max()) * 255  
    return magnitude.astype(np.uint8)
```

Annexe 24: Vérification

```
def compare_texts_aligned(real_text, predicted_text):
    """
    Compares two texts character by character with offset management.
    Displays similarity and clear visualization of errors.
    """
    real_text = real_text.strip()
    predicted_text = predicted_text.strip()

    i, j = 0, 0
    match = 0
    result = ""

    while i < len(real_text) and j < len(predicted_text):
        if real_text[i] == predicted_text[j]:
            result += real_text[i]
            match += 1
            i += 1
            j += 1
        else:
            if i + 1 < len(real_text) and real_text[i + 1] == predicted_text[j]:
                result += f"[+{real_text[i]}]"
                i += 1
            elif j + 1 < len(predicted_text) and real_text[i] == predicted_text[j + 1]:
                result += f"[-{predicted_text[j]}]"
                j += 1
            else:
                result += f"[{real_text[i]}≠{predicted_text[j]}]"
                i += 1
                j += 1

    if i < len(real_text):
        result += f"[+{real_text[i:]}]"
    if j < len(predicted_text):
        result += f"[-{predicted_text[j:]}]"

    total = max(len(real_text), len(predicted_text))
    similarity = round((match / total) * 100, 2)

    print(f"\n similarity : {similarity}%")
    print(" Differences :")
    print(result)
    print("\nLégende : [a#b] = wrong letter, [+a] = missing letter, [-b] = excess letter\n")
```


Annexe 25: Calcul de Luma l

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = [M] \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Obtention:

$$[M] = \begin{bmatrix} S_r X_r & S_g X_g & S_b X_b \\ S_r Y_r & S_g Y_g & S_b Y_b \\ S_r Z_r & S_g Z_g & S_b Z_b \end{bmatrix}$$

$$\begin{bmatrix} S_r \\ S_g \\ S_b \end{bmatrix} = \begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix}^{-1} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix}$$

Condition initiale:

Référence du blanc : E (1, 1, 1)

$$x + y + z = 1$$

$$\begin{aligned} x &= \frac{X}{X+Y+Z} \\ y &= \frac{Y}{X+Y+Z} \\ z &= \frac{Z}{X+Y+Z} \end{aligned}$$

Annexe 26: Calcul de Luma 2

	x	y
Rouge (Red)	0.7350	0.2650
Vert (Green)	0.2740	0.7170
Bleu (Blue)	0.1670	0.0090

$$\begin{aligned}
 x &= \frac{X}{X+Y+Z} \Leftrightarrow x(X+Y+Z) = X \quad L_1 \rightarrow \frac{L_1}{L_2} \\
 y &= \frac{Y}{X+Y+Z} \quad y(X+Y+Z) = Y \\
 &\Leftrightarrow \frac{x}{y} = \frac{X}{Y} \\
 &\quad y(X+Y+Z) = Y \\
 &\quad X = \frac{x}{y} Y \\
 &\Leftrightarrow y\left(\frac{x}{y} Y + Y + Z\right) = Y \\
 &\quad X = \frac{x}{y} Y \\
 &\Leftrightarrow xY + yY + yZ - Y = 0 \\
 &\quad X = \frac{x}{y} Y \\
 &\Leftrightarrow Y(x + y - 1) + yZ = 0 \\
 &\quad X = \frac{x}{y} Y \\
 &\Leftrightarrow Z = Y \frac{(1 - x - y)}{y}
 \end{aligned}$$